University “Politehnica” of Bucharest
Faculty of Electronics, Telecommunications, and Information Technology

DSA Semester Project

(Computers Programming Course - Semester III)

Student: Vlad Gabriel-Lucian

Group: 421G

• Introduction

Considering the fact that our tech products tend to get complicated as the days pass, people tend to go back to the roots of this era and look for simpler UIs and less complicated apps that just fulfill their basic purpose. The app I developed comes in handy for these types of applications. This electronic student database is suitable for primary and secondary school as its main purpose is to store information about each student (ID, name, grade, average grade and their inclination towards certain areas). What makes it suitable for primary school is the feature that indicates if a student is more likely to progress in areas like exact sciences and arts.

The app's main feature is searching for students, in a medium-sized database, by ID and providing all the information available for that student. It does that by utilizing a basic concept of Data Structures and Algorithms, a BST (Binary Search Tree). By utilizing a BST, it makes the app very fast and suitable for larger input data. The code also utilizes concepts of OOP, as working with classes could make the program a lot more scalable and easy to understand.

To be exact, OOP is utilized in storing the data for each student (a struct type) while the BST is used for accessing the input data as fast and organized as possible.

• Description of the program in natural language

This program serves as a student management system designed to efficiently handle, analyze, and display student data using a Binary Search Tree (BST). It allows users to store detailed information about each student, including a unique ID, full name, and grades for various subjects, organizing this data hierarchically based on student IDs. By leveraging the BST structure, the program ensures efficient operations, such as inserting new students, searching for specific records, and traversing all records in sorted order.

One of the key features of the program is its ability to calculate important academic metrics for each student. It computes their overall average grade, category-specific averages for subjects grouped into sciences or arts, and determines their academic inclination—whether they excel in sciences, arts, both (all-rounder), or show no particular specialization. This detailed analysis helps categorize students based on their performance.

Users can interact with the program to search for specific students by their ID, retrieving detailed information about them, including their grades, average grade, and academic inclination. Additionally, the program can display all students stored in the BST, presenting them in ascending order of their IDs.

The data is loaded dynamically from an external text file, which allows the program to work with different datasets without requiring changes to the code. This dynamic approach, combined with its interactive features and efficient data handling through the BST, makes the program scalable and suitable for managing large student datasets. Its modular design also ensures that new features, such as a graphical interface, can be added in the future without significant modifications.

- # Program Structure

The program is built around two main components: the Student structure and the StudentBST class, along with their associated methods and a TreeNode struct for tree organization. These components collectively manage the functionality of the program, which involves organizing, processing, and displaying student information.

1. **The Student Structure**
   The Student structure serves as a blueprint for storing individual student information and includes methods for processing their data.

a. **Attributes** (all public):

- int id: Unique identifier for the student, automatically assigned.
- string firstName: Contains the first name of the student.
- string lastName: Contains the last name of the student.
- vector<pair<string, double>> grades: Stores the student's grades for various subjects as subject-grade pairs.

b. **Methods**:

- **double calculateAverage() const**: Computes the overall average grade for the student.
- **double calculateCategoryAverage(const vector<string>& subjects) const**: Calculates the average grade for a specific group of subjects.
- **string getFullName() const**: Combines the first and last names into a full name.
- **string determineInclination() const**: Determines the student's academic inclination based on their grades, categorizing them as:
  - "Inclined towards Exact Sciences."
  - "Inclined towards Arts."
  - "All-rounder."
  - "No specific inclination."

2. **The TreeNode Structure**
   A supporting structure that represents a node in the Binary Search Tree (BST).
   a. **Attributes**:
   - Student stud: Stores a single Student object.
   - TreeNode* left: Pointer to the left child node.
   - TreeNode* right: Pointer to the right child node.

b. **Constructor**:

- Initializes a new node with a given Student object.

3. **The StudentBST Class**
   The StudentBST class is responsible for managing the Binary Search Tree and providing the core functionalities of the program.

a. **Attributes**:

- TreeNode* root: Pointer to the root of the BST.

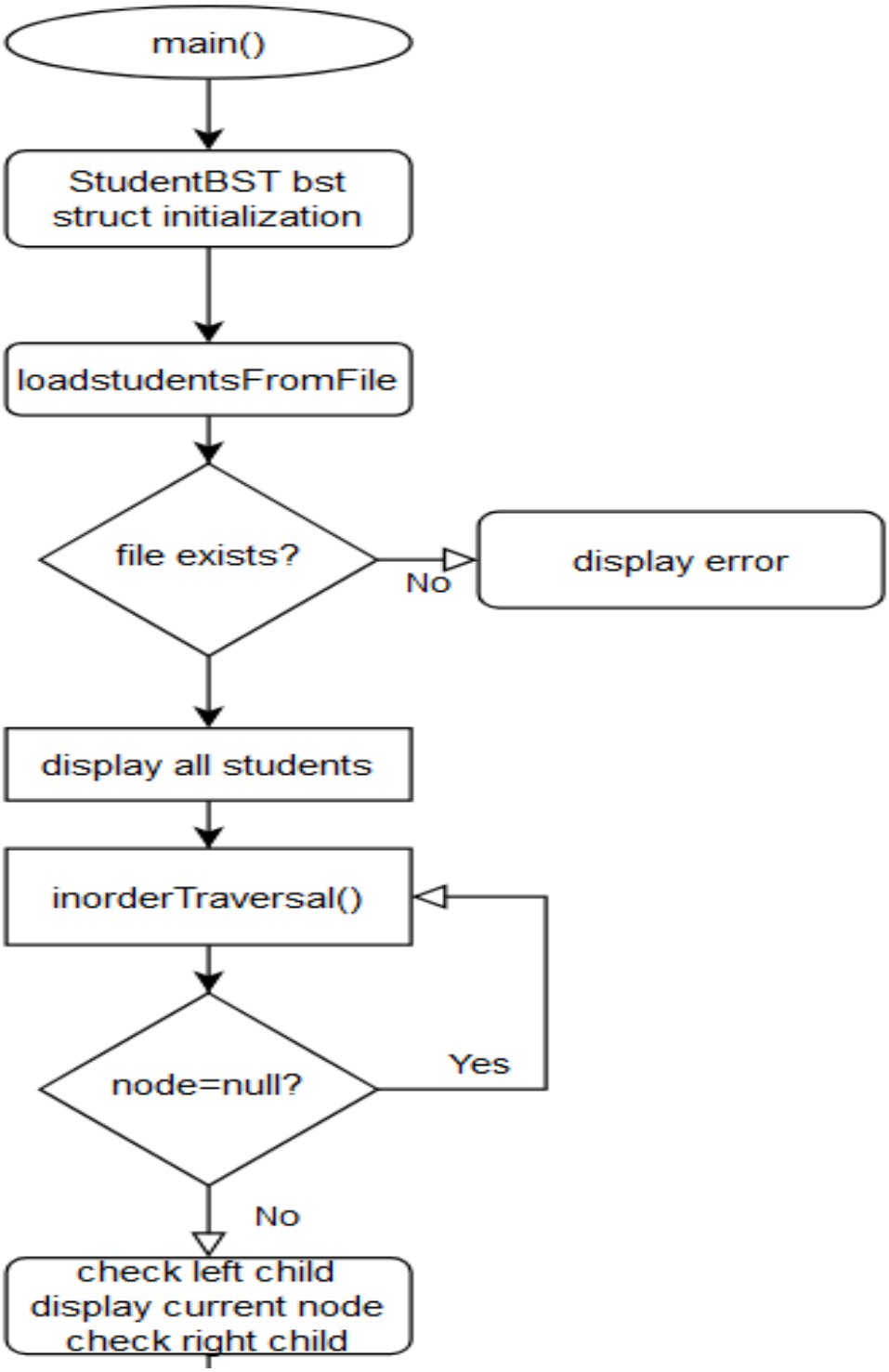
b. **Methods**:

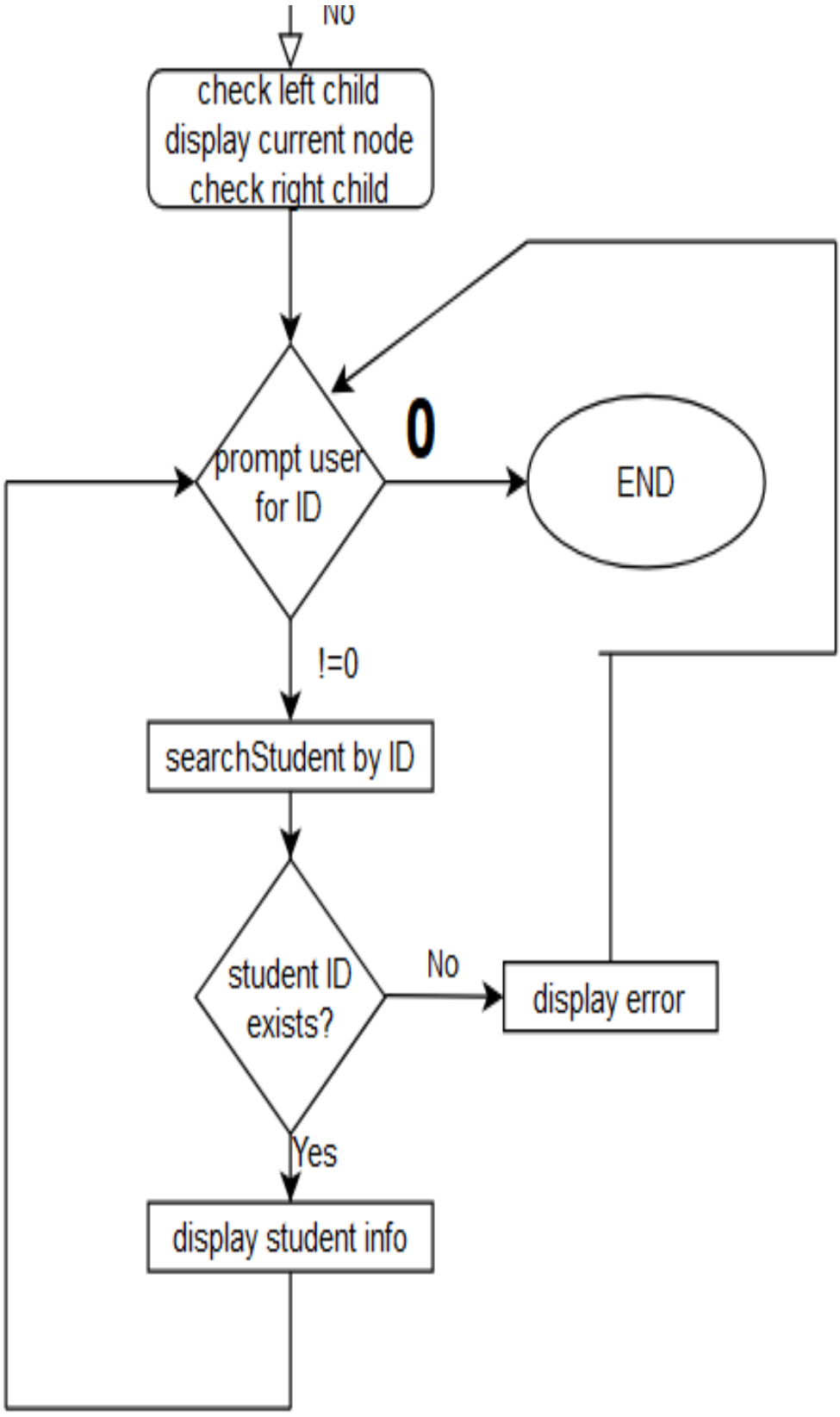
- **Private Methods**:
  - TreeNode* insert(TreeNode* node, const Student& student): Inserts a student into the BST.
  - TreeNode* search(TreeNode* node, int id) const: Searches for a student by their ID.
  - void inOrderTraversal(TreeNode* node) const: Traverses the tree in order and displays all students.
- **Public Methods**:
  - StudentBST(): Constructor to initialize an empty BST.
  - void insertStudent(const Student& student): Inserts a student into the BST.
  - void searchStudent(int id) const: Searches for a student and displays their details.
  - void displayAllStudents() const: Displays all students in the BST in sorted order.
  - void input() const: Interacts with the user to search for students by ID.

4. **ReadFromFile Function**
   - **void loadStudentsFromFile(const string& filename, StudentBST& bst)**: Reads student data from an external text file and inserts it into the BST.

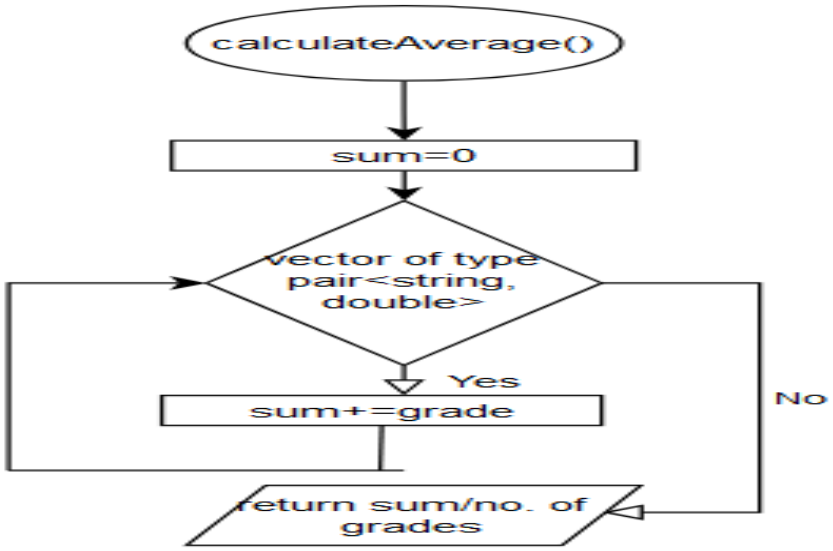
5. **The main() Function**
   a. Creates an instance of the StudentBST class.
   b. Loads student data from a file (student.txt).
   c. Displays all students stored in the BST.
   d. Runs the user interaction logic, allowing searches by student ID and displaying detailed information.

- Flowcharts for the crucial functions
  - main() function

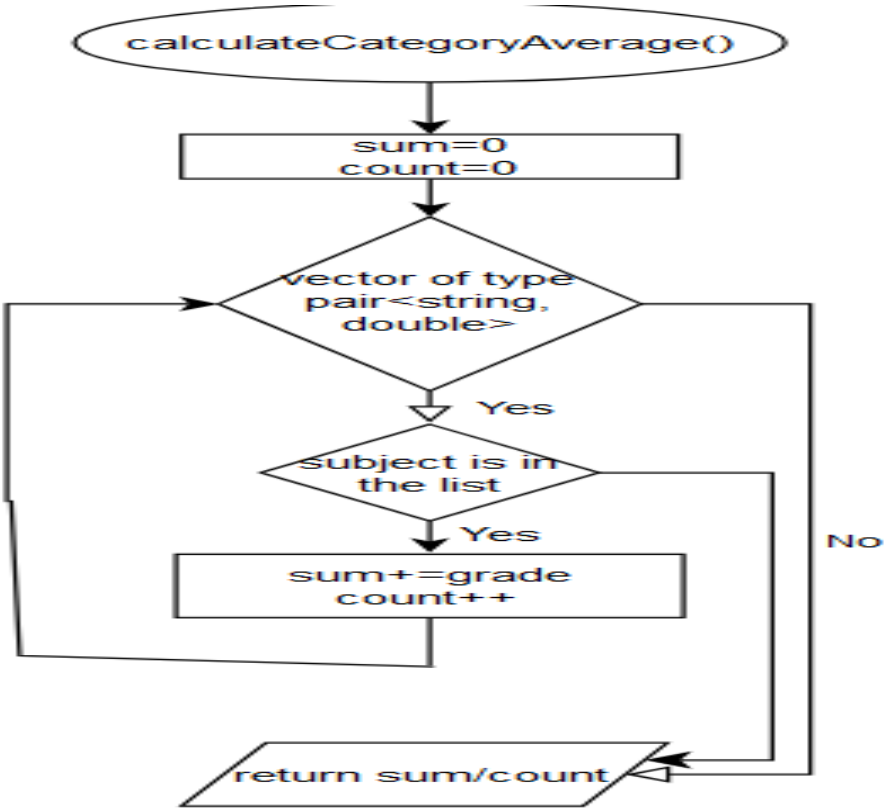



NO
check left child
display current node
check right child
prompt user for ID
0
END
!=0
searchStudent by ID
student ID exists?
No
display error
Yes
display student info

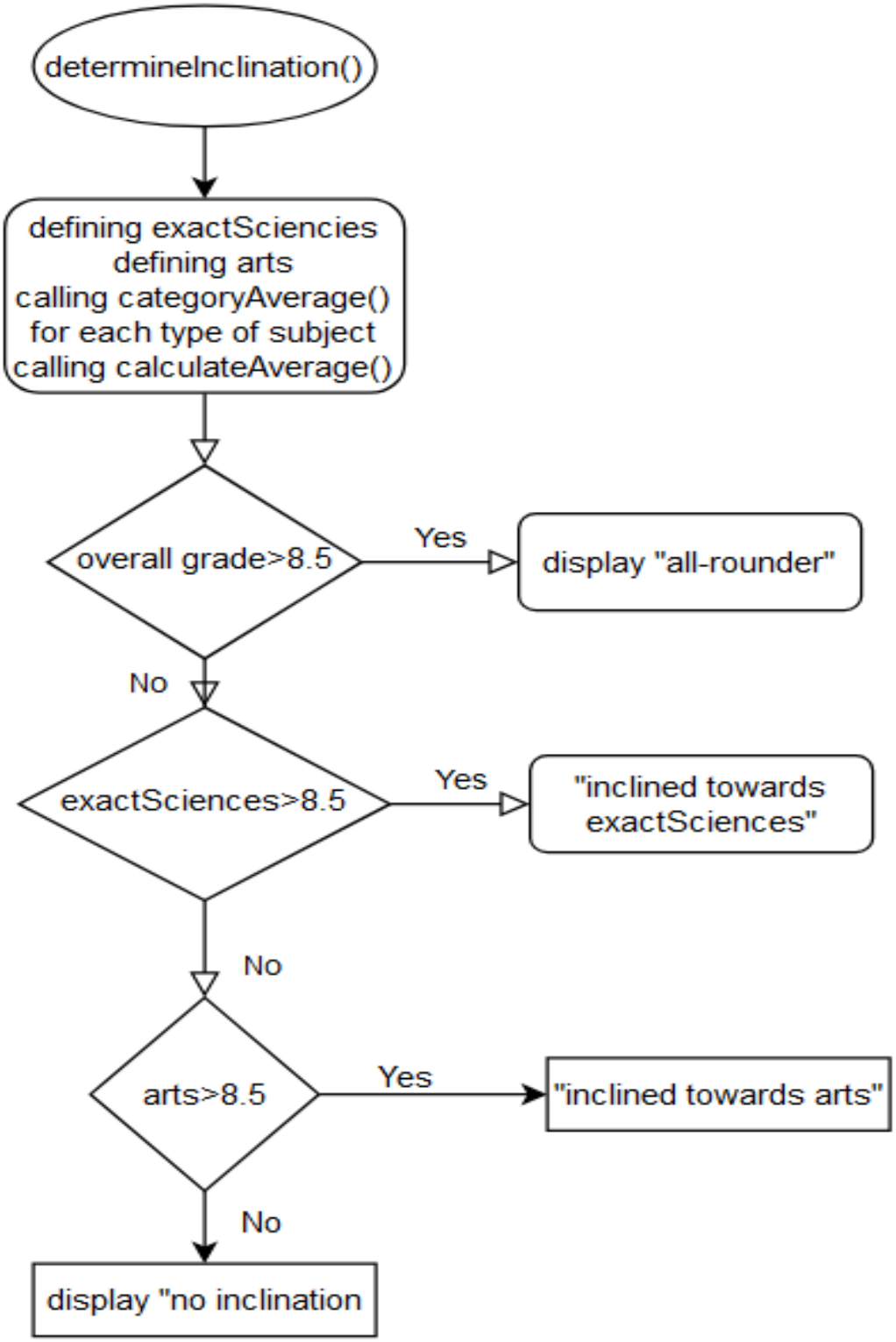
- calculateAverage() function



- calculateCategoryAverage() function



- determineInclination() function



- Listing for the entire program

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <fstream>
#include <sstream>
using namespace std;

struct Student {                                    //creating a structure to store all the info regarding the students
    int id;
    string firstName;
    string lastName;
    vector<pair<string, double>> grades;            //creating a vector pair that contains the name of the subject and its grade, data is
                                                    //stored as following: grade={"Math", 6.5}
    double calculateAverage() const {               //creating the function that computes the average grade for each student
        double sum = 0;
        for (const pair<string, double>& grade : grades) {      //iterating as long as there are elements in the vector
            sum += grade.second;
        }
        return sum / grades.size();                             //computing the average grade
    }

    double calculateCategoryAverage(const vector<string>& subjects) const {     //creating the function that computes the category average grade
        double sum = 0;
        int count = 0;
        for (const pair<string, double>& grade : grades) {                      //iterating through all elements from the pair vector 'grades'
            for(const string& subject : subjects) {                             //iterating through all the elements from the string vector 'subject'
                if (subject == grade.first) {                                   //checks if the subjects are part of the pair vector
                    sum += grade.second;                                        //ads the grade to the total sum
                    count++;                                                    //the number of subjects that are taken into consideration increments
                }
            }
        }
        return sum/count;                                                       //returns the average grade
    }

    string getFullName() const {                                                //function for grabbing the full name of the student from .txt file
        return lastName + " " + firstName;
    }

    string determineInclination() const {                                       //function that determines student's inclination based on their grades at
        vector<string> exactSciences = {"Math", "Physics", "Chemistry", "Biology", "Geography"};        //specific subjects
        vector<string> arts = {"Literature", "History", "Psychology"};

        double exactAverage = calculateCategoryAverage(exactSciences);
        double artsAverage = calculateCategoryAverage(arts);
        double overallAverage = calculateAverage();
        if(getFullName()=="BURCESCU Dan-Stefan"){
            return "DOG";
        }                                                                       //based on the avg. grade at certain subjects the student gets a recognition
        if (overallAverage > 8.5) {
            return "All-rounder";                                               //if the total avg. grade exceeds 8.5, then the student can be considered
        } else if (exactAverage > 8.5) {                                        //an "all-rounder"
            return "Inclined towards Exact Sciences";
```

```
        } else if (artsAverage > 8.5) {
            return "Inclined towards Arts";
        } else{
            return "No specific inclination";
        }
    }
};

struct TreeNode {                                                   //a struct type that is a node in the tree
    Student stud;                                                  //object that contains students' info
    TreeNode* left;
    TreeNode* right;
    TreeNode(const Student& s) : stud(s), left(nullptr), right(nullptr) {}   //constructor that has its object "Student" that will be part of the Tree
};

class StudentBST {
private:
    TreeNode* root;                                                 //pointer to the root of the tree

    TreeNode* insert(TreeNode* node, const Student& student) {      //method for inserting a node into the tree based on the ID of the current node
        if (node == nullptr) return new TreeNode(student);          //and the node to be introduced
        if (student.id < node->stud.id) {
            node->left = insert(node->left, student);
        } else if (student.id > node->stud.id) {
            node->right = insert(node->right, student);
        }
        return node;
    }

    TreeNode* search(TreeNode* node, int id) const {                //method used for searching students by ID
        if (node->stud.id == id) return node;
        if (id < node->stud.id) return search(node->left, id);
        if (id > node->stud.id) return search(node->right, id);
    }

    void inOrderTraversal(TreeNode* node) const {                   //method used for traversing the whole tree in order (left subtree, root, right subtree)
        if (node == nullptr) return;                                //this method is used for displaying all the students (nodes) in the tree
        inOrderTraversal(node->left);
        cout << "ID: " << node->stud.id << ", Name: " << node->stud.getFullName()<<"\n";
        inOrderTraversal(node->right);
    }

public:
    StudentBST() : root(nullptr) {}                                 //initializing the tree, the root being empty at first

    void insertStudent(const Student& student) {                    //method for inserting students into the tree
        root = insert(root, student);
    }
```

```
    void searchStudent(int id) const {                          //method used in searching for a student based on the ID
        TreeNode* result = search(root, id);
        if (result) {
            cout << "Student found: " << result->stud.getFullName() << " (ID: " << result->stud.id << ")\n";    //when the student is found the program
            for (const pair<string, double>& grade : result->stud.grades) {                                    //prompts the user with all the
                cout << "  Subject: " << grade.first << ", Grade: " << grade.second << "\n";                    //available data such as ID, Full Name,
            }                                                                                                   //grades for each subject, avrage grade
            cout << "  Average Grade: " << fixed << result->stud.calculateAverage() << "\n";                    //and inclination. this data is grabbed
            cout << "  Inclination: " << result->stud.determineInclination() << "\n";                           //from the methods used previously
        } else {
            cout << "Student with ID " << id << " not found!\n";           //if the ID is not part of the BST, the user will be prompted
        }
    }

    void displayAllStudents() const {                               //method used for displaying all the students by traversing the tree in order
        inOrderTraversal(root);
    }

    void input() const {                                    //method used for collecting the input data from the user
        while (true) {
            int id;
            cout << "Enter student ID to search (or 0 to exit): ";
            cin >> id;
            if (id == 0) {
                break;
            }
            searchStudent(id);
        }
    }
};

void loadStudentsFromFile(const string& filename, StudentBST& bst) {    //function used for reading the file containing the students
    ifstream file(filename);
    if (!file) {
        cout << "Error: file " << filename << " does not exist\n";
        return;
    }
    string line;
    int idCounter = 1;
    while (getline([&]file, [&]line)) {
        stringstream ss(line);
        Student s;
        s.id = idCounter++;
        ss >> s.lastName >> s.firstName;
        string subject;
        double grade;
        while (ss >> subject >> grade) {
            s.grades.push_back( x: {[&]subject, [&]grade});
        }
        bst.insertStudent(s);
    }
```

```
    file.close();
}

int main() {
    StudentBST bst;
    string filename = "student.txt";
    loadStudentsFromFile(filename, bst);
    cout << "All students:\n";
    bst.displayAllStudents();
    bst.input();
    return 0;
}
```

SOURCE CODE

- Instances of running the program

```
ID: 319, Name: TRANDAFIR Adrian
ID: 320, Name: VOICU Mirela
ID: 321, Name: AVRAM Cosmin
ID: 322, Name: BARBU Raluca
ID: 323, Name: CALIN Victor
ID: 324, Name: DIMA Andreea
ID: 325, Name: EFTIMIE Mihai
ID: 326, Name: GHERGHINA Cristina
ID: 327, Name: ION Razvan
ID: 328, Name: LUCA Iulia
ID: 329, Name: MARIN Sorin
ID: 330, Name: NITA Adina
ID: 331, Name: OLTEANU Alexandru
ID: 332, Name: PAUN Corina
ID: 333, Name: ROMAN Dragos
ID: 334, Name: SZABO Mirela
ID: 335, Name: TANASE Ionut
ID: 336, Name: VINTILA Diana
ID: 337, Name: ALBU Rares
ID: 338, Name: BALAN Roxana
ID: 339, Name: COSTEA Marian
ID: 340, Name: DOBRE Camelia
ID: 341, Name: ENE Dorin
ID: 342, Name: FRATILA Alina
ID: 343, Name: GRIGORAS Andrei
ID: 344, Name: IACOB Simona
ID: 345, Name: LAZAR Marius
ID: 346, Name: MANEA Daniela
Enter student ID to search (or 0 to exit):
```

```
Student found: STANESCU Mihai (ID: 15)
  Subject: Math, Grade: 10
  Subject: Physics, Grade: 5
  Subject: Biology, Grade: 6
  Subject: Chemistry, Grade: 8
  Subject: History, Grade: 6
  Subject: Literature, Grade: 6
  Subject: Psihology, Grade: 5.5
  Subject: Geography, Grade: 10
  Average Grade: 7.062500
  Inclination: No specific inclination
Enter student ID to search (or 0 to exit):16

Student found: CRISTEA Cosmin (ID: 16)
  Subject: Math, Grade: 5.000000
  Subject: Physics, Grade: 5.000000
  Subject: Biology, Grade: 6.000000
  Subject: Chemistry, Grade: 5.000000
  Subject: History, Grade: 6.000000
  Subject: Literature, Grade: 6.500000
  Subject: Psihology, Grade: 5.000000
  Subject: Geography, Grade: 6.000000
  Average Grade: 5.562500
  Inclination: No specific inclination
Enter student ID to search (or 0 to exit):
```

```
Student found: STAN Marius (ID: 36)
  Subject: Math, Grade: 6.500000
  Subject: Physics, Grade: 7.500000
  Subject: Biology, Grade: 7.000000
  Subject: Chemistry, Grade: 7.000000
  Subject: History, Grade: 8.000000
  Subject: Literature, Grade: 7.500000
  Subject: Psihology, Grade: 8.000000
  Subject: Geography, Grade: 7.500000
  Average Grade: 7.375000
  Inclination: No specific inclination
Enter student ID to search (or 0 to exit):19

Student found: PAVEL Carla-Angelica (ID: 19)
  Subject: Math, Grade: 9.000000
  Subject: Physics, Grade: 9.500000
  Subject: Biology, Grade: 7.500000
  Subject: Chemistry, Grade: 9.000000
  Subject: History, Grade: 9.000000
  Subject: Literature, Grade: 10.000000
  Subject: Psihology, Grade: 10.000000
  Subject: Geography, Grade: 10.000000
  Average Grade: 9.250000
  Inclination: All-rounder
Enter student ID to search (or 0 to exit):
```

```
Student found: PAVEL Carla-Angelica (ID: 19)
  Subject: Math, Grade: 9.000000
  Subject: Physics, Grade: 9.500000
  Subject: Biology, Grade: 7.500000
  Subject: Chemistry, Grade: 9.000000
  Subject: History, Grade: 9.000000
  Subject: Literature, Grade: 10.000000
  Subject: Psihology, Grade: 10.000000
  Subject: Geography, Grade: 10.000000
  Average Grade: 9.250000
  Inclination: All-rounder
Enter student ID to search (or 0 to exit):21

Student found: POPESCU Ana-Maria (ID: 21)
  Subject: Math, Grade: 8.000000
  Subject: Physics, Grade: 7.500000
  Subject: Biology, Grade: 8.000000
  Subject: Chemistry, Grade: 8.500000
  Subject: History, Grade: 9.000000
  Subject: Literature, Grade: 9.500000
  Subject: Psihology, Grade: 8.500000
  Subject: Geography, Grade: 8.000000
  Average Grade: 8.375000
  Inclination: Inclined towards Arts
Enter student ID to search (or 0 to exit):
```

- # Conclsions

This program demonstrates the effective use of a Binary Search Tree (BST) for managing and processing student data. By organizing information hierarchically, it enables efficient insertion, retrieval, and traversal of records. With features like calculating academic averages, determining inclinations, and interactive searches, the program provides a comprehensive system for student management. Its modular design and dynamic data loading make it scalable and adaptable for future extensions, ensuring practicality and efficiency in handling larger datasets.

- # Bibliography

i. https://stackoverflow.com/questions/13035674/how-to-read-a-file-line-by-line-or-a-whole-text-file-at-once
ii. https://www.geeksforgeeks.org/how-to-create-vector-of-pairs-in-cpp/
iii. https://www.geeksforgeeks.org/cpp-binary-search-tree/
iv. All laboratory guides used throughout the semester